Cryptography Algorithms and Verilog Implementation
130330
Testing an Algorithm (Assignment 6)
Date: Sunday, May 10, 2026

Submission:	**By not more than 2 people together**
Deadline:	**As instructed by the lecturer – via moodle**
According to JCT's policy, if you violate any submission or deadline criteria you disqualify your assignment	

A. About this Exercise

In this assignment we will see what it takes to test an encryption algorithm. We will use the DES algorithm as an example, but our main goal is more general: methodologies and practices used for testing.

You will deal with:

1. Test vectors
2. Calculator
3. What to do when you have to a debug of a Verilog code of an encryption algorithm…
4. Test bench methodologies
5. "Stub" design

And more...

B. The Concepts

B.1 Test Vectors

Every encryption algorithm (and many other algorithms and designs as well) have something that is called "test vectors". In other words, there are lists of known inputs and known outputs. Namely: key, plaintext and ciphertext.

For example, here is a link to a set of test vectors given by NESSIE (however, **some web sites might not exist anymore**, so watch out...):

https://www.cosic.esat.kuleuven.be/nessie/testvectors/bc/des/Des-64-64.test-vectors

This information is given in a form similar to:

```
Test vectors -- set 1
=====================
Set 1, vector#  0:
                         key=8000000000000000
                       plain=0000000000000000
                      cipher=95A8D72813DAA94D
                   decrypted=0000000000000000
           Iterated 100 times=F749E1F8DEFAF605
          Iterated 1000 times=F396DD0B33D04244

Set 1, vector#  1:
                         key=4000000000000000
                       plain=0000000000000000
                      cipher=0EEC1487DD8C26D5
                   decrypted=0000000000000000
```

```
            Iterated 100 times=E5BEE86B600F3B48
           Iterated 1000 times=1D5931D700EF4E15

Set 1, vector#  2:

                          key=2000000000000000
                        plain=0000000000000000
                       cipher=7AD16FFB79C45926
                    decrypted=0000000000000000
            Iterated 100 times=C4B51BB0A1E0DF57
           Iterated 1000 times=B2D1B91E994BA5FF
```

For "fscanf" purposes it may be easier to covert that list to even less complicated format, such as:

```
8000000000000000 0000000000000000 95A8D72813DAA94D
4000000000000000 0000000000000000 0EEC1487DD8C26D5
2000000000000000 0000000000000000 7AD16FFB79C45926
```

Here is another link for DES test vectors:

DES_decrypt/des_test_vectors.txt at master · delhatch/DES_decrypt · GitHub

B.2 Calculator

It is essential to have a calculator to compute the results for such an algorithm. Such calculators can be achieved either by compiling the C/C++ source code given for that algorithm (see for example: https://github.com/aimxhaisse/des), or by writing a Matlab code if you like… Moreover, for industry standard algorithms you have such calculator online, for example:

https://www.emvlab.org/descalc

B.3 And What Do You Do for Debug?

Suppose that wrote a Verilog code for the algorithm and… it doesn't work properly, what should you do?

A good practice for debug is to inspect the outputs of known stages in the algorithm and compare to the C/C++ run, in which you are able to print these outputs by the C/C++ code, and compare to your simulation. Here is an example:

http://lpb.canb.auug.org.au/adfa/src/DEScalc/index.html

B.4 Test Bench Methodologies

A common practice is to **let the testbench compare** between the output of the design to the reference (expected) output given by the test vector and to let the testbench report the fail pass by using $display commands. It could be something like:

```
# time =   700000 | K = 8000000000000000 | P = 0000000000000000 | C = 95a8d72813daa94d | Result = 95a8d72813daa94d | PASS
# time =   740000 | K = 4000000000000000 | P = 0000000000000000 | C = 0eec1487dd8c26d5 | Result = 0eec1487dd8c26d5 | PASS
# time =   780000 | K = 2000000000000000 | P = 0000000000000000 | C = 7ad16ffb79c45926 | Result = 0eec1487dd8c26d5 | FAIL
```

B.5 Stub

Usually, when a design is developed, many design portions are not yet ready. The designer must use "something", a "dummy module" to have a simple behavior just to be able to develop the rest of the design.

Similarly, it is the same for testbench development, the design is not yet ready, but the verification engineer must prepare

the testbench and have "something", a "dummy design", so testbench can be checked and developed. In such cases engineer who develops the testbench also has to create a "quick and dirty" "stub" module…

C. Lab Work

C.1 Test Vector

1. Create a test vector, of more than 10 test cases, in the form of

```
8000000000000000 0000000000000000 95A8D72813DAA94D
4000000000000000 0000000000000000 0EEC1487DD8C26D5
2000000000000000 0000000000000000 7AD16FFB79C45926
…
```

Look for existing / pre-calculated test vectors over the internet or by using AI tools.

If the original vector contains information other than the key, plaintext and expected ciphertext, use an automatic method to extract the relevant information from the original vector (and include the script/program in your report).

2. Simulate the test vector with the test vector example. (Include the resulting simulation output in your lab report).

3. Extract the first 10 lines of the simplified test vector format, to use in the next stage (and include in your report)

C.2 DES Stub and Self Comparing Testbench

1. Write a DES stub as follows:

Signal	Direction	Description
clk	input	Stub's clock
reset	input	Stub's reset, active high asynchronous
key[63:0]	input	key input to the stub (As a stub, this signal does nothing… it just get into the stub)
data_in[63:0]	input	Plaintext input to the stub (As a stub, this signal does nothing… it just get into the stub)
start	input	When this signal is high, the stub: 1. Writes a value from a (read by "fscanf") to data_out[63:0], 2. Resets the ready signal for 1 clock cycle 3. Sets the ready signal
data_out[63:0]	output	Ciphertext output, read from an appropriate file that accompany with the stub
ready	output	1: data_out is ready 0: data_out is not ready See above description for the start signal

This stab shall read it's output from a file. That file shall contain the top 10 ciphertext values of the above vector, such as

```
95A8D72813DAA94D
0EEC1487DD8C26D5
7AD16FFB79C45926
…
```

2. Based on the example given in class, expand the testbench to be able to
   - Drive the DUT
   - Drive the load signal
   - Let the testbench compare the results of the stub to the expected results that are given to the testbench, so the testbench could report provide a “PASS” or “FAIL” for each vector
   - Simulate the stub and let see that the stub pass all 10 vectors

3. Corrupt the results in **3 lines of the stub input file**. Run simulation and see that it reports on FAIL appropriatly.

D. Your Lab Report Must Include

1. Items required explicitly in section C.1
2. The Verilog code and testbench for the “stub”
3. Waveform and a log file of the “PASS” simulation (C.2, 2)
4. Waveform and a log file of the “FAIL” simulation (C.2, 3)